

RGS v1, byte by byte

A complete decoder reference

RGS is a lossy PCM16 audio format for game assets. Its compressed frames carry their own predictor state, and decoding a sample takes an integer prediction, a residual lookup, and a small state update. A slice can use either 3-bit or 2-bit residual codes, allowing the encoder to trade precision for space without adding another decoding algorithm.

This article walks through the complete v1 representation and decoding rules. The [RGS v1 format specification](#) is the normative authority; this companion explains that contract rather than defining another version. The [design article](#) discusses the choices behind the format, and the [reference implementation](#) is a single-header C codec.

RGS derives its four-tap LMS predictor and quantizer foundation from Dominic Szablewski's Quite OK Audio. His [QOA specification article](#) is a useful model for describing a small codec directly. RGS has its own headers, packing, and validation rules; RGS and QOA files are not interchangeable.

The units of the format

A **sample frame** contains one signed 16-bit value for each channel. For stereo, 100 sample frames means 200 PCM values. The sample counts in RGS headers are always per channel.

A **slice** represents up to 20 consecutive samples of one channel. A **compressed frame** represents up to 5,120 sample frames, so it contains at most 256 slices per channel. The compressed payload groups slices by channel; decoded PCM is interleaved in timeline order.

Property	RGS v1 limit
PCM representation	Signed 16-bit integers
Channels	1-8
Stored sample rate	1-44,100 Hz
File sample frames per channel	1-4,294,967,295
Sample frames per compressed frame	1-5,120
Samples per slice, per channel	1-20
Integer byte order	Little-endian

Every compressed frame except the last contains exactly 5,120 sample frames. The last contains the remaining samples, including a full 5,120 when the file length is an exact multiple. Empty files and unknown-length streams are not part of v1. Input above 44.1 kHz must be filtered and resampled before encoding; the codec itself does not resample.

File headers and predictor state

A file begins with twelve bytes

All offsets below are in bytes. Integer fields are unsigned unless described as predictor history or weights.

File offset	Bytes	Meaning
0	4	Magic: ASCII rgs!, or hex 72 67 73 21
4	4	Total sample frames per channel, nonzero
8	1	Version: exactly 01
9	1	File flags: exactly 07
10	2	Reserved: 00 00

The three file-flag bits advertise support for mixed-width frames (01), all-2-bit frames (02), and channel-planar payloads (04). They describe the v1 grammar, not an inventory of which modes a particular file uses. A file containing only 3-bit slices still stores 07. Other flag bytes and version numbers are rejected.

Channel count and sample rate appear in the first compressed frame. They remain constant throughout the file. A twelve-byte header on its own is therefore insufficient: at least one complete compressed frame is required.

Each frame starts with its own state

The frame header is eight bytes:

Frame offset	Bytes	Meaning
0	1	Low nibble: channel count; high nibble: frame flags
1	3	Sample rate, little-endian 24-bit integer
4	2	Sample frames in this compressed frame
6	2	Total compressed frame size, including header and state

Only three flag combinations are valid:

High nibble	Mode	Slice representation
4	Planar fixed	Every slice occupies 8 bytes and uses 3-bit codes
5	Planar mixed	A mode map chooses 8-byte or 6-byte slices
6	Planar all-2-bit	Every slice occupies 6 bytes and uses 2-bit codes

For example, the first byte of a stereo mixed frame is 52: flags 5 in the high nibble, two channels in the low nibble. Planar layout is mandatory. The mixed and all-2-bit bits cannot both be set; the remaining flag bit is reserved.

After the header come 16 bytes of state for each channel, in channel order:

```
channel 0: history[0..3], weights[0..3]
channel 1: history[0..3], weights[0..3]
...
```

Each array contains four little-endian signed 16-bit integers. Interpret an unsigned word $u \geq 32768$ as $u - 65536$. History runs from oldest at index 0 to newest at index 3.

Load this state at the beginning of every frame. Do not reconstruct it from the preceding frame: an encoder may choose a different initial predictor. State then evolves continuously across that channel's slices until the frame ends. Slice boundaries do not reset the predictor.

Channel layout and slice bits

Finding the slices

For a frame with N sample frames and C channels, define:

$S = \text{ceil}(N / 20)$	slices per channel
$M = \text{ceil}(S / 8)$	mode-map bytes per channel
$B = 8 + 16 * C$	frame header plus all predictor states

In fixed mode, slices start at frame offset B , and the exact frame size is $B + 8 * S * C$. In all-2-bit mode, slices also start at B , and the size is $B + 6 * S * C$.

A mixed frame first stores all C mode maps at offset B , followed by all slice payloads. Each channel gets M map bytes. Bit $s \% 8$ of byte $s / 8$ selects that channel's slice s : zero means 3-bit codes in eight bytes; one means 2-bit codes in six bytes. Unused high bits in the last map byte are zero, and a checked decoder rejects a map that violates this rule.

The mixed frame's exact size is:

$B + M * C + \text{sum}(\text{size of each selected slice in every channel})$

All slices of channel 0 precede all slices of channel 1. Within each channel, slices run in timeline order. Mode maps are grouped together before the payloads; they are not interspersed with them.

Consider a final stereo frame containing 41 sample frames. It has three slices per channel, covering 20, 20, and 1 sample. Maps 05 and 02 give:

header:	8 bytes
state 0, state 1:	32 bytes
map 0 = 05, map 1 = 02:	2 bytes
channel 0 slices:	6 + 8 + 6 bytes
channel 1 slices:	8 + 6 + 8 bytes
total:	84 bytes

At 44.1 kHz, this frame's header is 52 44 ac 00 29 00 54 00. The final one-sample slice still occupies its full six or eight bytes.

Mixed maps may select the same width for every slice. The reference encoder removes such redundant maps, but a decoder does not require both widths to occur in a mixed frame.

Bits inside a slice

Load an 8-byte slice as a little-endian unsigned 64-bit integer. For a 6-byte slice, load exactly six bytes and zero-extend the result; do not read two bytes past the slice boundary.

Both layouts put the scalefactor index in the low four bits, followed by residual codes in sample order:

Field	3-bit slice	2-bit slice
Scalefactor index	Bits 0-3	Bits 0-3
Sample 0 code	Bits 4-6	Bits 4-5
Sample 1 code	Bits 7-9	Bits 6-7
Sample 19 code	Bits 61-63	Bits 42-43
Fixed padding	None	Bits 44-47, zero

Codes and dequantization

Extracting codes and padding

For code width b , the extraction rule is:

```
scale    = word & 15
code[j]  = (word >> (4 + b*j)) & ((1 << b) - 1)
```

The four high padding bits of every 2-bit slice are zero, including full slices. A checked decoder rejects nonzero padding. Separately, a partial slice has code positions beyond its actual sample count. Writers should zero those unused codes, but decoders ignore them and do not update the predictor for them.

The code widths are not the complete storage cost. A full 3-bit slice uses 64 bits for 20 samples, or 3.2 bits per sample; a full 2-bit slice uses 48, or 2.4 bits per sample. Frame state, headers, and any mode maps add overhead.

Turning codes into residuals

The scalefactor selects a row of four positive magnitudes:

Scale	A	B	C	D
0	1	3	5	7
1	5	18	32	49
2	16	53	95	147
3	34	113	203	315
4	63	210	378	588
5	104	345	621	966
6	158	528	950	1477
7	228	760	1368	2128
8	316	1053	1895	2947
9	422	1405	2529	3934
10	548	1828	3290	5117
11	696	2320	4176	6496
12	868	2893	5207	8099
13	1064	3548	6386	9933
14	1286	4288	7718	12005
15	1536	5120	9216	14336

A 3-bit code selects from [+A, -A, +B, -B, +C, -C, +D, -D]. A 2-bit code selects from [+A, -A, +C, -C]. These integer values define the residuals exactly; no floating-point scale calculation is needed in the decoder.

Reconstruction and worked example

Predict, reconstruct, update

For each residual r , use the current channel's four history values and weights in this order:

```
p = floor((w[0]*h[0] + w[1]*h[1] + w[2]*h[2] + w[3]*h[3]) / 8192)
x = clamp(p + r, -32768, 32767)
d = floor(r / 16)

for i = 0..3:
    w[i] += h[i] < 0 ? -d : d

h[0] = h[1]
h[1] = h[2]
h[2] = h[3]
h[3] = x
```

Emit x as the next PCM value for that channel. In a frame's interleaved output buffer, sample t of channel c goes at $\text{output}[t * C + c]$, where t continues across all of that channel's slices.

Three details affect every independent implementation:

- Both divisions use mathematical floor. For example, $\text{floor}(-1/16)$ is -1 , while C integer division would produce zero.
- Update weights using the old history, then shift history and append the **clamped** output sample. Do not use the input PCM or an unclamped value.
- History and weights are 16-bit values when loaded from a frame, but weights can grow during decoding. Do not truncate them after an update. Use wide intermediates for products and sums, and avoid signed overflow; signed 64-bit arithmetic is sufficient for v1's limits.

Each channel has its own state. Switching between 2-bit and 3-bit slices changes the residual lookup, not the predictor or update rule.

A complete 42-byte example

This file contains one mono frame at 44.1 kHz and produces 20 PCM values. The left column is the hexadecimal file offset:

```
0000: 72 67 73 21 14 00 00 00 01 07 00 00
000c: 61 44 ac 00 14 00 1e 00
0014: 00 00 00 00 00 00 00 00
001c: 00 00 00 00 00 e0 00 40
0024: be 51 41 41 10 01
```

The frame byte 61 selects one channel and all-2-bit mode. Its declared size is 001e, or 30 bytes: eight bytes of header, sixteen of state, and one six-byte slice. Histories are all zero; weights are $[0, 0, -8192, 16384]$.

The slice's low nibble is e, selecting scale 14. Its first two codes are 3 and 2, so their residuals are -7718 and $+7718$. The first prediction is zero and the first output is -7718 . Its update uses $\text{floor}(-7718/16) = -483$, making the weights $[-483, -483, -8675, 15901]$. The second prediction is -14981 , giving $-14981 + 7718 = -7263$.

The complete decoded sequence is:

```
-7718, -7263, -7238, -5140, -3578,
-2966, -3562, -2834, -685, 227,
-255, 492, 2443, 2859, 4283,
6685, 7429, 8956, 8876, 9542
```

This example is covered by the [decoder regression tests](#) and can be reproduced directly with the arithmetic above. It specifies what these bytes decode to; it does not require a current encoder to choose these same bytes when encoding audio.

Validation and integration boundaries

Validation is part of decoding

A checked decoder verifies the complete object before reporting whole-file success:

1. Check magic, version, exact file flags, reserved bytes, and nonzero sample count. Read channel count and sample rate from a complete first frame and enforce their limits.
2. Require every subsequent frame to retain that channel count and rate. Require its sample count to equal $\min(5120, \text{remaining file samples})$.
3. Check the flag combination, availability of all predictor states and mode maps, and zero padding in maps and 2-bit slices.
4. Compute the exact payload size from its modes. Require it to equal the frame-size field and fit within the available input bytes.
5. Accumulate sample counts and byte offsets without overflow. After the last declared sample, require exact end-of-file; trailing bytes are invalid.
6. Ensure the destination can hold $\text{file_samples} * \text{channels}$ PCM16 values, using arithmetic that cannot wrap when computing that capacity.

Ignoring unused codes in a partial slice does not permit ignoring map padding, 2-bit padding, or extra bytes at the end of a file. RGS has no checksum, so structural validity alone does not detect every possible corruption.

In the reference API, `rg_rgs_read_header` validates the file header and the complete first frame. It does not certify later frames or final EOF. `rg_rgs_decode_s16` performs whole-file checked decoding. The trusted decoder is intended only for the same complete, immutable byte stream after successful checked decoding.

What the encoder may change

The wire format defines reconstruction, not the search used to select codes. The reference encoder uses high, medium, and low presets. High writes 3-bit slices; medium and low may also write 2-bit slices. It measures decoded error and can retry difficult channels with other predictor states or width choices. The chosen state is written into the frame header, so these decisions require no decoder-side heuristic.

An encoder improvement may change both compressed bytes and reconstructed PCM while remaining a valid v1 encoder. Preset names are not a guarantee of quality ordering for every signal. Encoding options and bitrate hints are not extra file fields. Measurements and listening guidance belong in the [performance and quality notes](#), separate from the decoding contract.

RGS also leaves seeking indexes, loop regions, speaker layouts, and other game metadata to an outer container or application. A frame's stored state lets a decoder reconstruct it once its location and stream metadata are known, but the reference streaming API provides sequential decoding and reset rather than a seek index. See [streaming integration](#) for buffer ownership, worker decoding, and callback use.